

AI-Based PR Test Impact Analyzer

Intelligent Test Coverage Identification from Pull Request Changes

Prepared by:

Engineering Productivity Team

Audience:

Engineering Leadership / CTO Office

Date:

June 2026

Version:

1.0 — Initial Proposal

EXECUTIVE SUMMARY

Today, when a developer raises a pull request, quality assurance teams have no automated way to determine which existing automated test scenarios are affected by those code changes. This forces QA engineers to manually review PRs, trace code paths, and decide which test suites to execute — a process that is slow, error-prone, and does not scale as our codebase and test coverage grow.

This proposal outlines the design, architecture, and implementation plan for an AI-Based PR Test Impact Analyzer — an intelligent, self-service tool that automatically identifies affected BDD test scenarios from a given pull request. The system is designed as a reusable, organization-wide platform that any team can integrate with their own development and test repositories.

60–80% <i>Reduction in manual test selection effort</i>	< 2 min <i>Average time to identify affected test scenarios</i>	Org-wide <i>Reusable across all dev & test teams</i>
---	--	--

PROBLEM STATEMENT

Current State & Pain Points

Our engineering organization maintains two separate repositories: a development repository built on .NET and ColdFusion with SOAP-based services, and a test automation repository built in C# using Selenium, SpecFlow/ReqNroll, NUnit, and Gherkin BDD scenarios. The disconnect between these two codebases creates a critical visibility gap:

- **Manual PR review for test selection:** QA engineers must read every PR, understand code context, and manually map changes to test scenarios. This takes 30–90 minutes per PR on average.
- **No semantic mapping between code and tests:** Test scenarios are written in business language (Gherkin). There is no automated linkage between a changed .NET class or ColdFusion component and the human-readable test scenarios that exercise it.
- **Over-testing and under-testing:** Without precise impact analysis, teams either run the entire regression suite (wasting CI/CD pipeline time) or miss affected scenarios (shipping defects).
- **Does not scale:** As the codebase grows, the problem compounds. A team of 5 QA engineers today cannot manually triage PRs for a team of 50 developers.
- **Repeated work across teams:** Every project team solves this problem independently — or more commonly, does not solve it at all.

Business Impact

The absence of automated PR-to-test mapping results in measurable business costs:

- Delayed release cycles due to conservative full-suite regression runs.
- Escaped defects reaching QA or production when impacted tests are missed.
- QA engineer time wasted on low-value manual triage instead of test improvement.
- Longer feedback loops for developers — slowing down the PR review and merge cycle.
- Inconsistent quality standards across teams with no shared tooling.

PROPOSED SOLUTION

The PR Test Impact Analyzer is a C# / ASP.NET Core web application that accepts a pull request reference, a development repository path, and a test repository path. It analyzes the PR diff, builds an understanding of what code changed, indexes the test scenario library, and uses AI reasoning to identify which BDD scenarios are affected by those changes. Results are displayed on a clean web interface within minutes.

How It Works — End to End

- 1. User Input** Developer or QA engineer navigates to the web UI, enters the PR number/URL, specifies paths to the dev repository and test repository (or selects from connected repos), and clicks Analyze.
- 2. PR Diff Extraction** The system connects to the dev repository and extracts the full diff for the PR — all changed files, added/removed lines, and modified symbols — using LibGit2Sharp.
- 3. Static Code Analysis (Phase 1)** The diff is processed by technology-specific analyzers. For .NET code, Microsoft Roslyn parses the C# AST to identify changed classes, methods, and API controller actions. For ColdFusion, a purpose-built regex parser extracts changed CFC components and functions. For SOAP, a WSDL diff parser identifies modified service operations.
- 4. Test Scenario Indexing (Phase 2)** The test repository is scanned for all .feature files. Each Gherkin scenario is parsed (name, tags, steps, associated step definition code) and stored in an in-memory index. This index is cacheable across analyses for the same test repo state.
- 5. AI Impact Reasoning (Phase 3)** A structured change summary (not raw code) is sent to an LLM with the test scenario index. The LLM reasons over which scenarios exercise the changed functionality and returns a ranked list of impacted scenario names with explanations.
- 6. Results Display** The web UI presents the list of affected test scenario names, their feature files, associated tags, and the AI's reasoning for why each was flagged. Results can be exported to CSV or consumed via API.

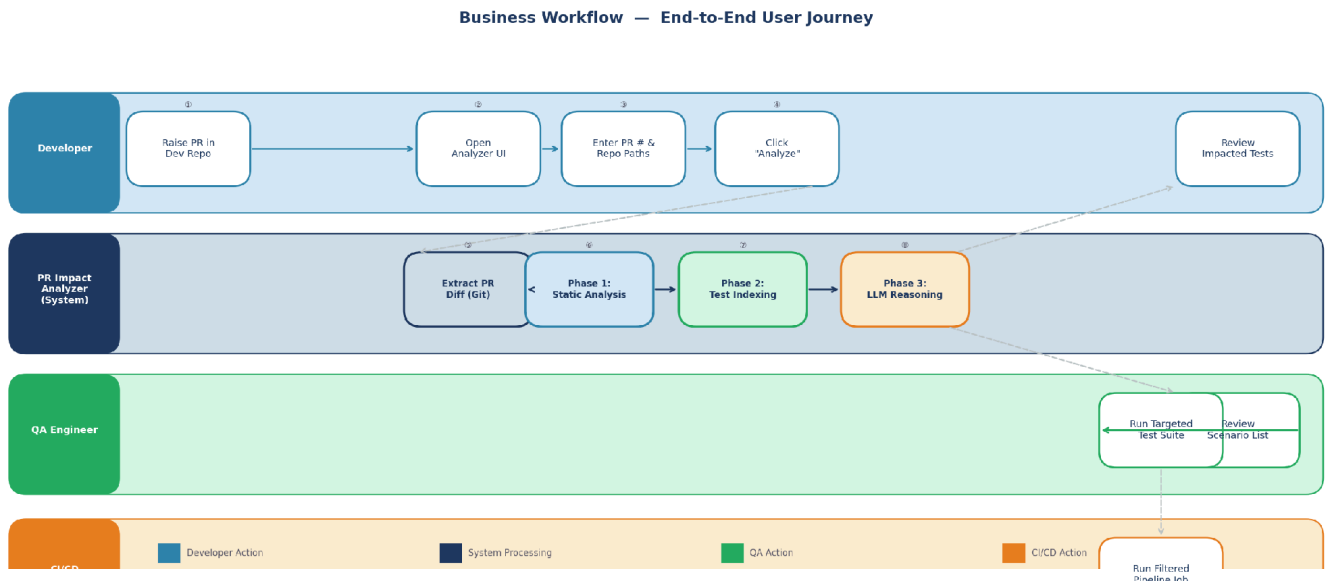


Figure 1 — End-to-End Business Workflow: from PR raise to targeted test execution

APPROACH OPTIONS & RECOMMENDATION

Five implementation approaches were evaluated. The table below summarizes each with its feasibility, cost, and accuracy profile.

Approach	Description	Accuracy	Cost/Complexity	Verdict
① Static Keyword Matching	Regex/string match PR diff against test files	Low	Very Low	Rejected
② Embedding + Vector Search	RAG: embed tests + PR diff, cosine similarity search	Medium	Medium	Alternative
③ LLM Brute Force	Send entire codebase + tests to LLM in one prompt	High*	Very High	Rejected
④ Hybrid: Static + LLM	Roslyn AST extraction → structured context → LLM reasoning	High	Low–Med	RECOMMENDED
⑤ Agentic (Semantic Kernel)	AI agent navigates repos autonomously with tool calls	Highest	High	Advanced Mode

* LLM Brute Force accuracy is high only for very small repos; context window limits render it impractical for real-world use.

RECOMMENDED ARCHITECTURE: HYBRID STATIC + LLM

The recommended approach (Option 4) combines deterministic static analysis with AI semantic reasoning. This design was chosen because it delivers the highest accuracy at the lowest per-analysis cost, is explainable to end users, and scales to large repositories without context window limitations.

Why This Approach Was Selected

- **Semantic gap bridged by AI:** Gherkin scenarios use business language; code uses technical identifiers. No deterministic system can reliably map between them. LLM reasoning bridges this gap precisely.
- **Cost-efficient token usage:** By extracting a compact structured summary via Roslyn before calling the LLM, we reduce input tokens by 90%+ compared to sending raw code. This makes every analysis fast and cheap.
- **Deterministic pre-filtering:** Static analysis produces a reliable, auditable list of what changed. The AI then reasons over this — not raw code. This hybrid gives us the best of both worlds.
- **Extensibility by design:** ICodeAnalyzer and ITestParser interfaces allow any team to plug in their own tech stack — Java, Python, Playwright, Cucumber — without changing core logic.
- **Explainability:** The system shows users not just which tests are affected, but why — which changed API or component triggered each test's inclusion.

Visual Solution Architecture

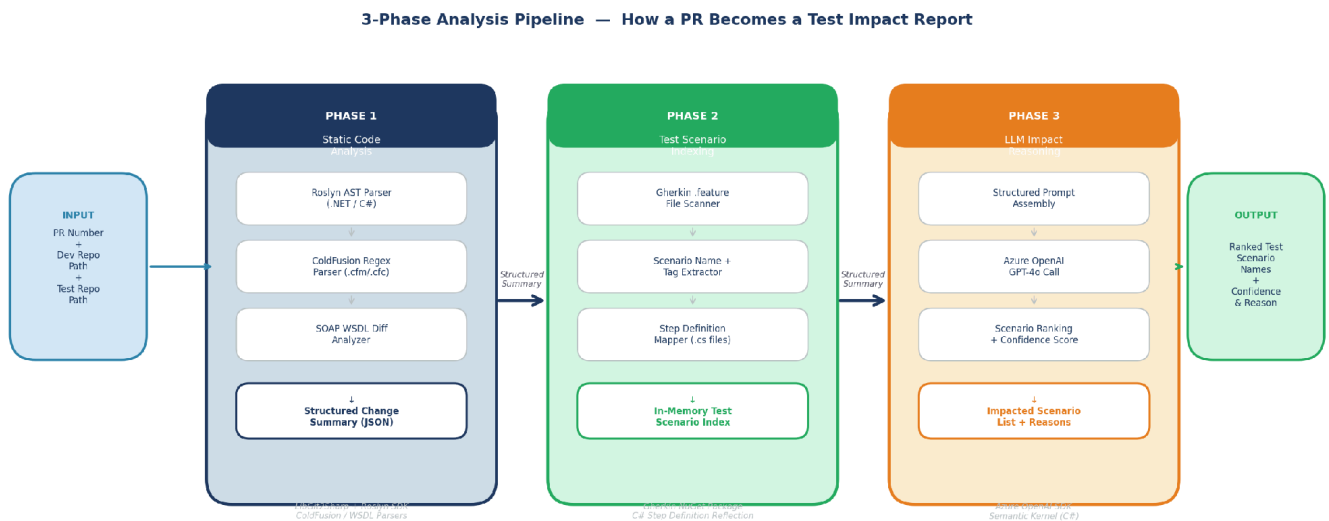


Figure 3 — 3-Phase Analysis Pipeline: PR Diff → Static Analysis → Test Indexing → LLM Reasoning → Results

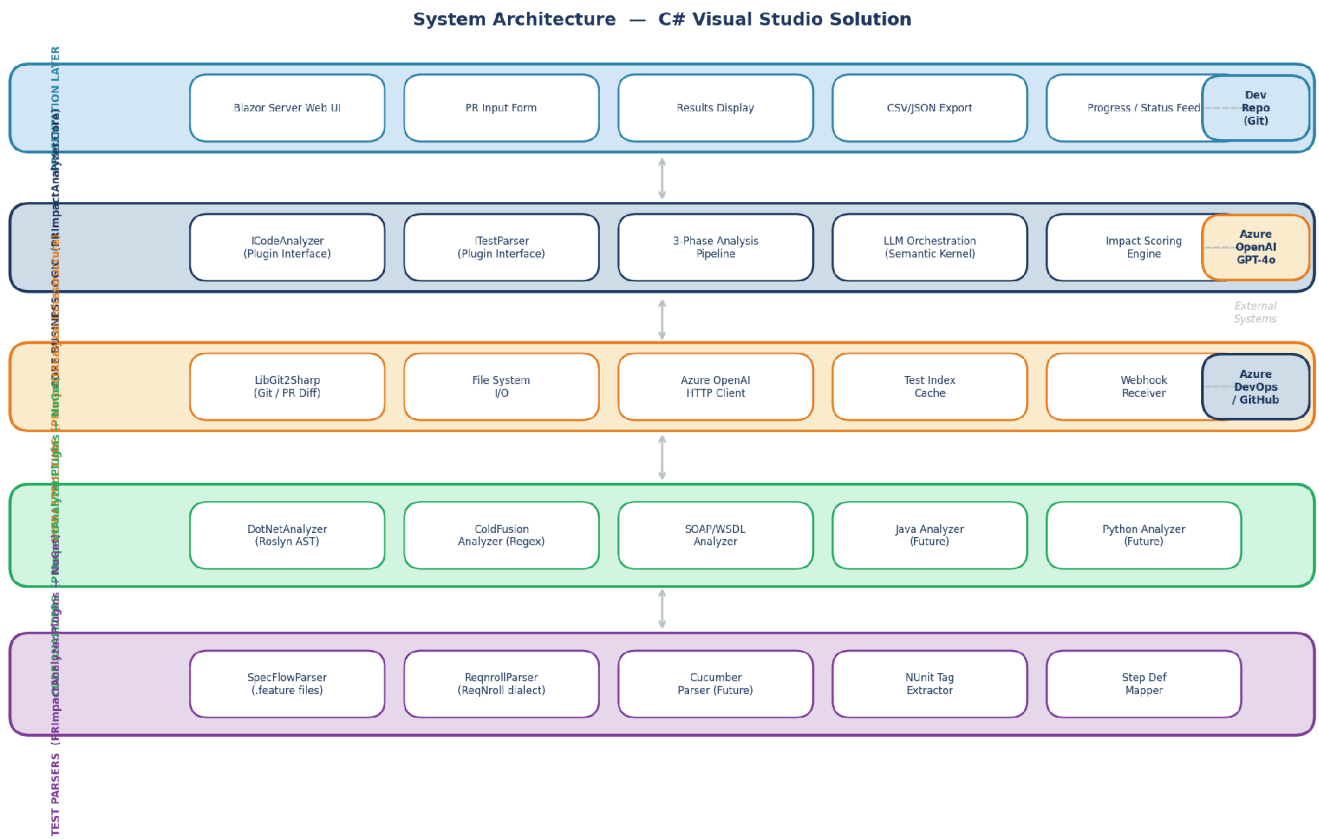


Figure 2 — Layered System Architecture: 4 C# projects with plugin interfaces and external integrations

C# Visual Studio Solution Structure

The solution is organized into four projects following clean architecture principles:

- **PRImpactAnalyzer.Web** ASP.NET Core Blazor Server — the single-page web UI. Users enter PR details, trigger analysis, and view results. No JavaScript framework required.
- **PRImpactAnalyzer.Core** Business logic. Contains the ICodeAnalyzer and ITestParser plugin interfaces, the three-phase analysis pipeline, LLM orchestration via Azure OpenAI SDK or Semantic Kernel, and all domain models.
- **PRImpactAnalyzer.Infrastructure** External concerns: LibGit2Sharp for repository access, file system I/O, HTTP clients for LLM APIs, and optional vector store integration for caching embeddings.
- **PRImpactAnalyzer.Plugins** Technology-specific adapters published as NuGet packages. Includes DotNetAnalyzer (Roslyn), ColdFusionAnalyzer (regex), SoapWsdIAnalyzer, SpecFlowParser, and ReqrollParser. Teams install only the packages they need.

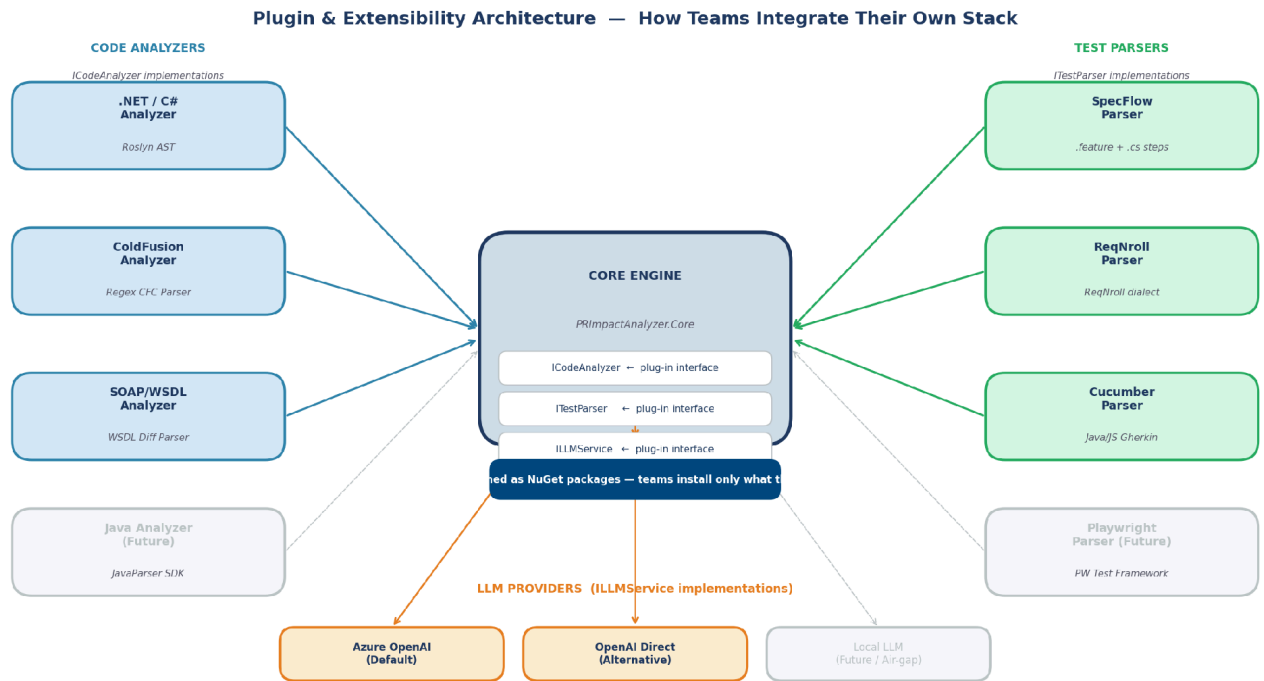


Figure 4 — Plugin & Extensibility Architecture: any team integrates by implementing *ICodeAnalyzer* or *ITestParser*

Key Technology Choices

Concern	Technology	Rationale
Language / Platform	C# / .NET 8	Matches org standard; strong tooling
Web Framework	ASP.NET Core Blazor Server	Full C# stack; real-time UI updates
.NET Code Parsing	Microsoft Roslyn SDK	Official C# compiler APIs; production-grade AST
Git Integration	LibGit2Sharp	Native .NET git library; no shell dependency
Gherkin Parsing	Gherkin NuGet package	Official Cucumber parser; handles all dialects
AI Reasoning	Azure OpenAI / Semantic Kernel	C# native; GPT-4o; Semantic Kernel for agentic mode
Project Template	NuGet plugin packages	Teams install adapters for their tech stack only

IMPLEMENTATION ROADMAP

The project is structured in three phases over approximately twelve weeks. Each phase delivers a working, usable increment — not just design artifacts.

Phase 1 — Foundation (Weeks 1–4)

Goal: Working end-to-end prototype with .NET + SpecFlow support

- Project scaffold: solution, 4 projects, DI setup, plugin interfaces
- LibGit2Sharp integration: PR diff extraction from local repo paths
- Roslyn-based DotNetAnalyzer: extracts changed classes, methods, endpoints
- SpecFlowParser / ReqnrollParser: parses all .feature files + step definitions
- Basic Blazor Server UI: input form, loading state, results list
- Azure OpenAI integration: structured prompt + scenario ranking
- End-to-end demo with a real PR from dev repo

Phase 2 — Production Hardening (Weeks 5–8)

Goal: Production-quality tool with ColdFusion + SOAP support and org deployment

- ColdFusionAnalyzer: regex-based CFC/function parser for .cfm and .cfc files
- SoapWSDLAnalyzer: WSDL diff parser for changed service operations
- Test index caching: avoid re-indexing unchanged test repos
- Confidence scoring: rank test scenarios by impact likelihood
- CSV/JSON export: results consumable by CI/CD pipelines
- Docker container packaging: deploy anywhere
- Internal developer documentation and onboarding guide

Phase 3 — Org-Wide Platform (Weeks 9–12)

Goal: Self-service platform with GitHub/Azure DevOps integration and agentic mode

- Azure DevOps / GitHub PR webhook: auto-trigger analysis on PR creation
- Agentic Expert Mode: Semantic Kernel agent for complex multi-layer PRs
- NuGet package publishing: teams install only the adapters they need
- CI/CD pipeline integration: output test scenario list as pipeline artifact
- Admin portal: team onboarding, repo configuration, usage analytics
- Org-wide rollout with pilot teams from 3 different product areas

RESOURCE REQUIREMENTS

Resource	Quantity	Duration	Notes
----------	----------	----------	-------

Senior C# / .NET Developer	1 FTE	12 weeks	Core engine, Roslyn integration, LLM orchestration
QA / Automation Engineer	0.5 FTE	12 weeks	Test parser validation, scenario accuracy review
DevOps Engineer	0.25 FTE	Weeks 8–12	Container, CI/CD pipeline integration, deployment
Tech Lead / Architect	0.25 FTE	12 weeks	Plugin interface design, code review, org rollout planning
Azure OpenAI Service	Pay-per-use	Ongoing	Estimated \$50–150/month at 1,000 PR analyses/month
Azure DevOps / GitHub Access	Existing	—	PR webhook and repo read access required

COST-BENEFIT ANALYSIS

Investment Estimate (12-Week Build)

Based on standard engineering rates and Azure OpenAI pricing:

Development Cost (1.5 FTE × 12 weeks)	~\$72,000 – \$90,000
Azure OpenAI (LLM usage, first year)	~\$1,200 – \$2,400
Infrastructure (container hosting, Azure)	~\$2,000 – \$4,000/year
Total First-Year Investment	~\$75,000 – \$97,000

Quantified Benefits

Assuming 50 developers raising 10 PRs/week each and 5 QA engineers at fully-loaded cost of \$120,000/year:

- Manual QA triage time saved (60 min/PR → 5 min/PR): **~\$180,000/year in QA engineer time**
- Reduction in unnecessary full-suite regression runs (est. 40% CI cost saving): **~\$30,000–60,000/year in compute**
- Defect escape reduction — earlier, targeted test execution: **Qualitative; estimated 15–25% reduction in escaped defects**
- Org-wide reuse — zero incremental build cost for teams 2–N: **Compounding ROI as more teams adopt**

Estimated ROI: Break-even within 5–6 months. Year-1 net benefit: \$110,000–\$150,000.

RISK ANALYSIS & MITIGATION

Risk	Probability	Impact	Mitigation
LLM produces false positives (flags tests not actually affected)	Medium	Medium	Confidence scoring + feedback loop. Users can mark results as correct/incorrect to fine-tune prompts over time.
LLM misses affected tests (false negatives)	Low–Medium	High	Conservative prompting strategy: instruct LLM to err on the side of inclusion. Phase 2 adds confidence thresholds.
ColdFusion parsing is incomplete (complex CFScript)	Medium	Medium	Start with common patterns; add regex coverage iteratively. ColdFusion changes can be flagged for manual review as fallback.

API costs exceed budget at high PR volume	Low	Low	Token-efficient structured prompts keep cost under \$0.05/analysis. Caching test index further reduces per-PR cost.
Developer adoption resistance	Low	Medium	Phase 1 demo with real PRs from dev team. Quick feedback loop builds trust. Optional CI/CD integration removes friction.
LLM vendor lock-in (Azure OpenAI)	Low	Low	ILLMService interface abstracts the LLM provider. Switching to another provider requires only a new adapter class.

SUCCESS METRICS & KPIS

The following metrics will be tracked from Phase 1 deployment onwards to measure the system's effectiveness and adoption:

KPI	Definition	Target	Measurement Cadence
Precision	% of flagged scenarios that are genuinely impacted	> 80%	Monthly, from user feedback
Recall	% of truly impacted scenarios that are correctly flagged	> 90%	Monthly, sampled validation
Analysis Latency	Time from PR submission to results displayed	< 2 minutes	Automated, per-analysis
Manual Triage Reduction	Reduction in time QA spends selecting tests per PR	> 60%	Quarterly survey
Adoption Rate	% of PRs analyzed using the tool (vs. total PRs)	> 70% within 6 months	Monthly dashboard
Escaped Defect Rate	Defects found in QA/production attributable to missed tests	Trending down	Monthly from bug tracker
Team Onboarding Time	Time for a new team to integrate their repos	< 1 day	Per new team onboarding

WHY BUILD VS. BUY

Several commercial test impact analysis tools exist (Launchable, Gradle Enterprise, etc.). However, none of them support our specific stack combination of .NET + ColdFusion + SOAP service layer with SpecFlow/ReqNroll BDD tests. They also cannot be extended to cover our internal frameworks. A build approach gives us:

- **Full stack coverage:** ColdFusion + SOAP parsing is a hard requirement no commercial tool supports.
- **Org-wide extensibility:** Plugin architecture lets any team add their tech stack. Commercial tools are rigid.
- **Data sovereignty:** No code leaves our environment (LLM calls use Azure OpenAI within our tenant).
- **Zero per-seat licensing:** One build cost, unlimited internal users forever.
- **Custom integration:** Deep Azure DevOps / internal CI/CD hooks that commercial tools do not support.

RECOMMENDED NEXT STEPS

To proceed, we request leadership approval and the following actions within the next two weeks:

- 1. Approve Phase 1 funding and resource allocation** Assign 1 senior C# developer + 0.5 QA engineer to the project for 12 weeks.
- 2. Designate a pilot team** Select one product team to serve as the Phase 1 pilot — ideally a team with active PR volume and good test coverage.
- 3. Provision Azure OpenAI access** Request an Azure OpenAI GPT-4o deployment within the organization's Azure tenant for LLM calls.
- 4. Set up project repository** Create a new internal repository for the solution under the engineering-productivity organization.
- 5. Schedule Phase 1 kick-off** 30-minute alignment call with engineering lead, pilot QA team, and the development team to align on repo access and success criteria.

This investment pays for itself within one quarter **and scales to the entire organization at zero marginal cost.**

Questions or feedback: Engineering Productivity Team | vishalmelmatti@gmail.com